Smart City Traffic Management with Multi-Agent Reinforcement Learning and Decision Networks

Group 2 · Project report · Reasoning and Decision Making under Uncertainty (MAIB DSC 103) · November 2026
Dashboard: https://smarttrafficu-mmzjikh5bg8frby3adzdc2.streamlit.app/ · Code and data: github.com/Sbot-c/Smart_Traffic_U

Summary

We built a traffic simulator of a 3 × 3 Downtown-Dubai-style grid in which traffic lights, autonomous vehicles (AVs) and buses are learning agents trained with Q-learning and SARSA, and a decision network that chooses a control strategy by weighing efficiency, safety and emissions under uncertainty. Across seven scenarios the RL packages cut delay against a fixed-time plan by 41 % in rush hour and 49 % in an arena event surge. Separating the effects shows that **cooperative AV routing is the main lever** (–24 % in rush hour under the same signals). A fair-comparison study then showed that RL signals **match, but do not beat, an actuated controller tuned on validation runs**: they reach expert-level control without manual tuning. A pedestrian max-wait safety supervisor makes RL as safe as actuated control at extreme demand. The decision network recommends actuated signals with cooperative routing in 6 of 7 scenarios, consistent with these findings.

1. System design

Each agent has a state, a set of actions and a reward that already contains the safety and emission objectives, so they are learned rather than added afterwards. All controllers pass the same safety shield (minimum green, pedestrian clearance, maximum green, emergency-vehicle pre-emption, bus priority).

Agent	State	Actions	Reward (minimised)
Traffic lights (9)	Phase, green time, queues on green / red, arrivals, pedestrian or spill-back flag (2,250 states)	Keep or change phase every 5 s	Delay + 0.5 × pedestrian delay + conflicts (1 = 300 s) + CO_2
Autonomous vehicles	Moves left, congestion on the two candidate links (64 states)	Next link (cooperative routing); speed advisory and lane selection (extensions)	Travel time + delay imposed on others
Buses	Schedule deviation, load, headway, signal ahead, cross queue (144 states)	Go, request priority, hold 20 s	Passenger-minutes + schedule deviation + priority cost

2. Simulation and learning protocol

The simulator models a peak-demand ramp and a three-lane corridor, pedestrian crossings with waiting and jaywalking, emergency-vehicle pre-emption, rain and sandstorm effects on capacity, speed and risk, and buses with stops, loads and a timetable. Seven scenarios are evaluated: normal flow, rush hour, incident, rain, sandstorm, event surge and a bus-priority corridor. Agents are pre-trained on 280 episodes spanning all scenarios, adapted per scenario, and the general or adapted table is chosen on **validation** seeds. Every reported number comes from **8 held-out test episodes**, with identical arrivals for every controller (common random numbers). The main log has 15,120 rows (junction × 30 s window).

3. Decision network

The influence diagram has chance nodes for weather, traffic density and pedestrian flow, which drive congestion, collision risk and emissions, and through them a health-impact node (the congestion → emissions → health dependency). The decisions are signal timing (fixed, actuated, Q-learning, SARSA) and routing (static, cooperative RL). Three utility nodes score efficiency, safety and environment with default weights 0.40 / 0.35 / 0.25. The conditional probabilities are learned from 11,664 simulated junction × 10-minute samples. The dashboard adds value of information and weight sensitivity. With default weights the best expected utility is **actuated signals + cooperative routing** in six scenarios (rain: actuated + static routes).

4. Results

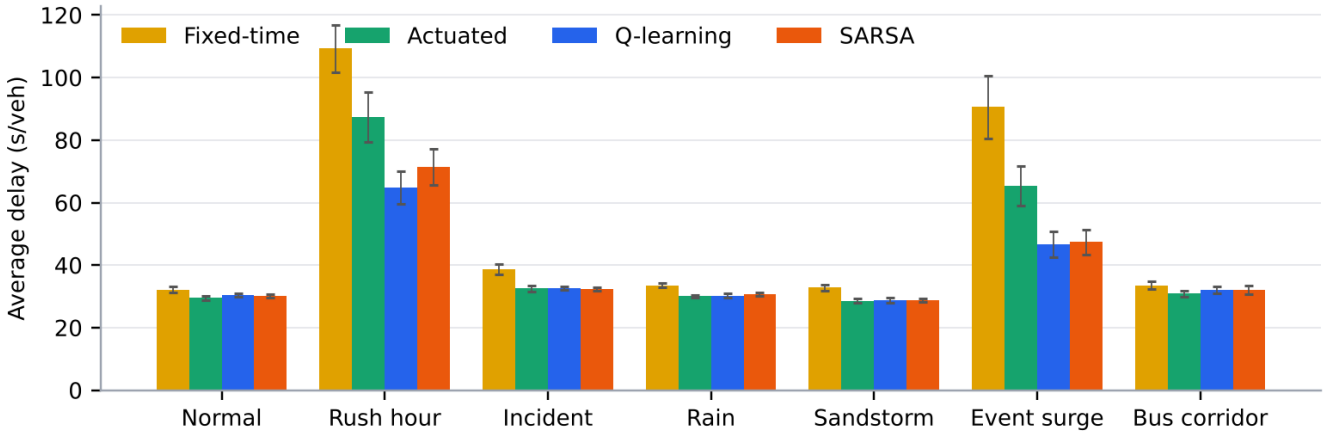


Figure 1. Average delay by scenario and policy package (mean ± s.d., 8 test episodes). Actuated = actuated signals + static routes; Q-learning / SARSA = RL signals + cooperative AV routing.

Gains concentrate where demand spikes (Figure 1). Against fixed-time, the Q-learning package cuts rush-hour delay from 109.3 to 64.8 s (–41 %) and event-surge delay from 90.6 to 46.6 s (–49 %). In normal, rain and sandstorm conditions every adaptive controller is within one to two seconds of the others. Because a package mixes signals and routing, Figure 2 holds routing fixed: routing alone takes actuated signals from 87.5 to 66.9 s, while Q-learning signals beat actuated signals by only 3 % with the same routing (p = 0.08, not significant).

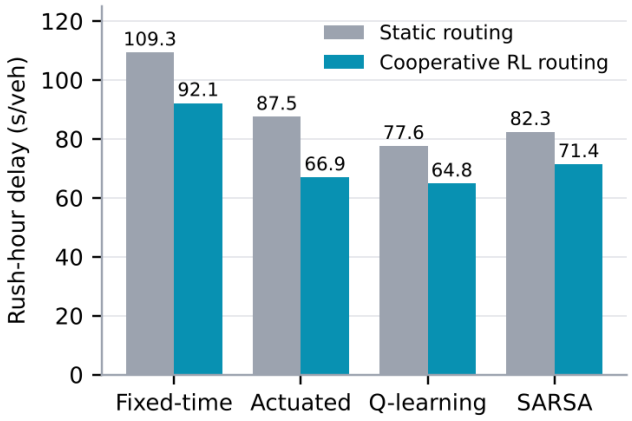


Figure 2. Rush hour: each signal controller with static and cooperative routing.

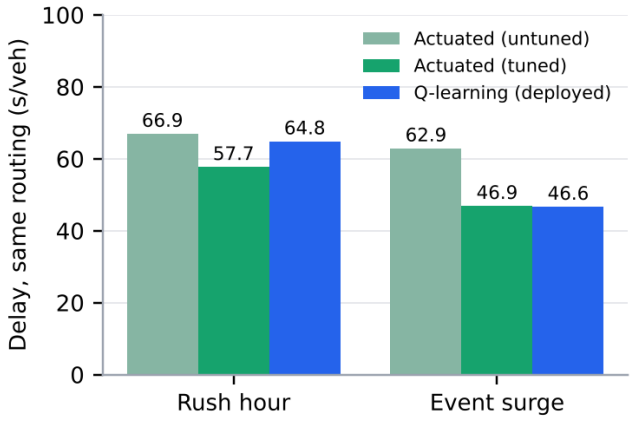


Figure 3. Same cooperative routing: actuated as first written, actuated tuned on validation runs, and Q-learning.

Fair comparison. The actuated rule had not been tuned, while RL had 280+80 training episodes. We tuned its four parameters (maximum green, queue ratio, queue offset, pedestrian service time) per scenario over 81 combinations on validation runs, then tested with the same routing (Figure 3). The event-surge advantage becomes a tie (46.6 vs 46.9 s) and in rush hour tuned actuated is 11 % faster (57.7 vs 64.8 s). Re-training Q-learning three times gives 72.5 ± 2.1 s in rush hour, so the deployed table is a good run. Our claim is therefore that RL learns tuned-expert performance on its own, and that cooperative routing is the gain that survives a fair test.

5. Research extensions: closing the review gaps

Five weaknesses were addressed with new experiments (*extensions.py*). They use the deployed tables, the same test episodes and switches that default to off, so the results above are unchanged.

Gap	What we added	Finding
AV speed control and lane selection	V2I speed advisory (ease off towards a red light, ≤ 15 s per link); AVs pass a vehicle waiting to turn or blocked by a full exit	Lane selection: AV delay –2.2 %, network delay –0.7 %. Advisory: fewer stops, conflicts (–1.1 %) and CO_2, for +6.8 % AV delay. Small at 25 % AVs; grows with AV share
Bus load balancing	Load at every stop, 95th-percentile load, passengers left behind; even-headway holding rule as benchmark	Rush-hour buses run full under every strategy: capacity, not balance, is binding. A bus every 3 min cuts left-behind passengers from about 35 to 1 (Figure 5). RL buses keep the best on-time record (66 % vs 57 %)
Safety is a proxy	Each conflict mechanism (red-light running, rear-end, jaywalking) halved and doubled; all policies re-run	Safest policy unchanged in 90 % of 42 cases; changes only swap the two non-learning controllers
Unfair baseline	Actuated tuned on validation runs; RL averaged over training runs	See Figure 3: tie in event surge, tuned actuated better in rush hour

Gap	What we added	Finding
RL safety at high demand	Pedestrian max-wait guard: serve a crossing once its first pedestrian has waited 22-30 s	At 1.6 × demand RL's extra conflicts were jaywalking (35 s pedestrian waits). Guard 30 s: conflicts 18.6 → 10.7 per 1,000 veh (actuated 10.6), delay 179 vs 205 s (Figure 4). Costs delay in normal rush hour; off during event surges

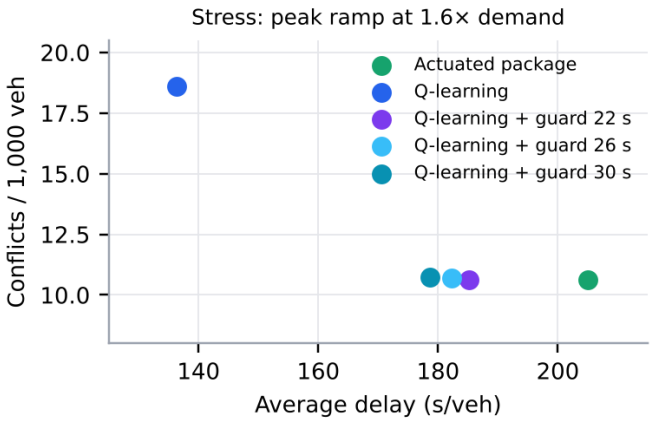


Figure 4. Delay against expected conflicts at 1.6× demand with the peak ramp; lower-left is better.

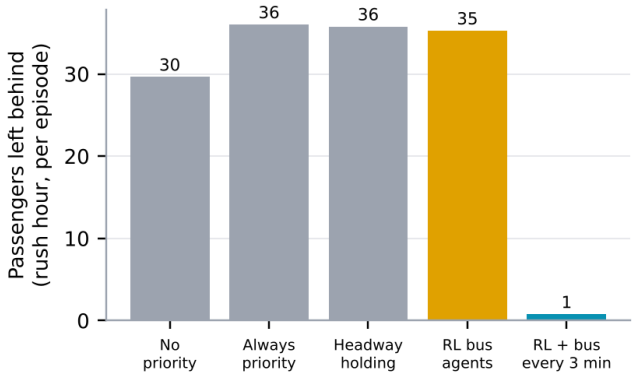
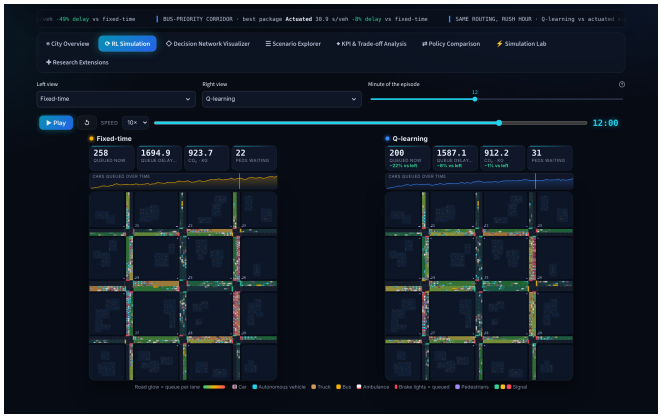


Figure 5. Passengers left behind by full buses in rush hour, by bus strategy.

6. Dashboard



The Streamlit command centre has eight views: City Overview (live replay, KPIs, network map), RL Simulation (side-by-side replay, agent decisions, Q-tables), Decision Network Visualizer (influence diagram, expected utilities, value of information), Scenario Explorer, KPI & Trade-off Analysis (heatmaps, AHP + TOPSIS, value of time), Policy Comparison (signals × routing with statistics), Simulation Lab and Research Extensions. The **Simulation Lab** re-runs all four controllers live under eleven conditions plus the three extension add-ons, with linked sliders (rush hour raises demand and pedestrians; demand raises bus passengers; weather and crashes add emergency calls; a bus lane brings 3-minute service). It shows the simulation winner next to the decision network's recommendation, a live replay and a stress test.

Figure 6. Side-by-side replay in the dashboard: fixed-time (left) and Q-learning (right), same arrivals.

7. Recommendations and limitations

Recommendations. (1) Deploy cooperative AV routing first: it is the largest and most robust gain. (2) Use RL signals where hand-tuning is costly or conditions drift, because they reach tuned-expert performance unaided. (3) Always run a safety supervisor and switch the pedestrian guard by demand level. (4) Let the decision network pick the combination for the day's conditions and stakeholder weights. (5) In the peak, add bus capacity rather than more priority.

Limitations. Safety is measured as expected conflicts, a surrogate for crashes (the ranking is robust; absolute values are not crash rates). The value of time is a U.S. figure and NOx factors are order-of-magnitude values. The network is one synthetic 3 × 3 grid, not calibrated to observed Dubai counts, with an assumed 25 % AV share. Tabular RL suits this state size; deep RL would be needed for raw detector or camera inputs. Next steps are calibrating to RTA detector data, learning the speed advisory, and testing a larger network. All results can be regenerated from the repository (*run_experiments.py*, *robustness.py*, *extensions.py*).